

in golang, how to you handle that creating a instance of struct, even if you use named fields, compiler doesn't catch missing assignment to fields? Is creating verbose NewX function really the best option?

It is one of the most common gripes developers have with Go: by default, the language treats all unassigned struct fields as their "zero value" (`0` , `""` , `nil` , `false`). The compiler won't bat an eye if you completely forget a field.

While writing a verbose `NewX(...)` constructor function is the standard and most idiomatically accepted approach, it isn't the *only* way.

Here are the best strategies to handle this, ranging from native Go idioms to compiler-enforced tricks.

1. The Standard: The `NewX` Constructor (With a Twist)

You don't just have to write a basic `NewX` function; you can design it to leverage Go's type system to make omissions impossible. Instead of passing 10 arguments, you can use the **Functional Options Pattern** or force explicit config structs.

If you stick to a standard constructor, you shift the burden from "remembering fields" to "the compiler tells me I missed an argument":

Go



```
type User struct {
    ID    string
    Email string
}

// The compiler will force you to provide both arguments here.
func NewUser(id string, email string) User {
    return User{ID: id, Email: email}
}
```

2. The Linter Approach: `exhauststruct` (Highly Recommended)

If you want the compiler—or rather, your build pipeline—to catch missing fields during `struct` initialization, you should use a linter.

The Go ecosystem has a highly popular linter called `exhauststruct` (which is built into `golangci-lint`). It specifically checks whether all fields are initialized in structure literals.

If you have:

Go



```
user := User{ID: "123"} // Missing 'Email'
```

Running the linter will throw an error:

```
User is missing fields: Email
```

This gives you the best of both worlds: you can use clean, readable named-field initializations, and your CI/CD pipeline will break if someone forgets a field.

3. The Compile-Time Trick: Unexported Sentinel Fields

If you want to prevent users of your package from initializing a struct directly altogether (forcing them to use your `NewX` function), you can add a hidden, unexported field of an empty struct type.

Go



```
package accounts

type User struct {
    ID      string
    Email   string
    _       struct{} // The sentinel field
}
```

Why this works:

Because the blank field `_` cannot be named outside of the `accounts` package, developers in other packages *cannot* use named fields to initialize it anymore.

Writing `accounts.User{ID: "1", Email: "a@b.com"}` will cause a compile error because they didn't (and can't) initialize the hidden field. They are completely forced to use your constructor function.

4. Design Around the "Zero Value"

Go's philosophy heavily relies on the idea that the **zero value should be useful**. If your struct is failing because a field is missing, ask yourself if the struct can be redesigned so that its default state behaves safely.

For example, instead of requiring a `Timeout` duration field to be set to `10 * time.Second`, check for `0` in your methods and apply a default:

Go



```
func (c *Client) Fetch() {
    timeout := c.Timeout
    if timeout == 0 {
        timeout = 30 * time.Second // Safe fallback
    }
    // ...
}
```

Summary: Which should you choose?

Method	Catch Location	Effort	Best
<code>NewX()</code> Function	Compile-time	Medium	Public structs
<code>exhauststruct</code> Lint er	Lint-time / CI	Low	Intermediate code safety, boilerplate
Sentinel Field (<code>_ struct{}</code>)	Compile-time	Low	Strict consistency across

If you are working on a team or a large codebase, installing `golangci-lint` and enabling `exhauststruct` is generally considered the cleanest way to solve this without bloating your code with constructors.

how to configure exhauststruct linter

Configuring the `exhauststruct` linter is straightforward, especially if you use `golangci-lint`, which is the standard lint runner in the Go ecosystem.

Here is how you can set it up globally for your project or fine-tune it to skip certain structures.

Step 1: Create or Open Your `.golangci.yml` file

In the root directory of your Go project, look for or create a file named `.golangci.yml`.

Step 2: Enable the Linter

Add `exhauststruct` to your enabled linters list:

YAML



```
linters:
  enable:
    - exhauststruct
```

Step 3: Configure `exhauststruct` Behavior (Optional)

By default, `exhauststruct` will yell at you for every unassigned field in every struct. This can be annoying for external packages (like database drivers or HTTP servers) where you only care about setting a couple of fields.

You can configure rules under the `linters-settings` block to ignore specific

structs or entire packages:

YAML



```
linters-settings:
  exhauststruct:
    # If true, it will check structs from external dependencies as well.
    # Setting this to false prevents external library updates from breaking
    include-dependencies: false

    # Standard regular expressions to exclude specific types.
    exclude:
      # Exclude all structs ending with 'Config' or 'Options'
      - '.*Config$'
      - '.*Options$'
      # Exclude specific structs by exact name
      - 'net/http.Server'
```

Step 4: Run the Linter

Once configured, you can run the linter via your terminal:

Bash



```
golangci-lint run
```

If you have a struct initialized like this:

Go



```
type User struct {
    Name string
    Age  int
}

u := User{Name: "Alice"} // Missing 'Age'
```

The output will look similar to this:

Plaintext



```
main.go:10:7: User is missing fields: Age (exhauststruct)
```

Pro-Tip: Inline Exclusions

If you have a specific instance in your code where you intentionally want to leave fields to their zero-value and don't want to change your global YAML config, you can use a linter directive comment directly above the struct:

Go



```
//nolint:exhauststruct // We intentionally want default values for the remain  
config := User{  
    Name: "Bob",  
}
```